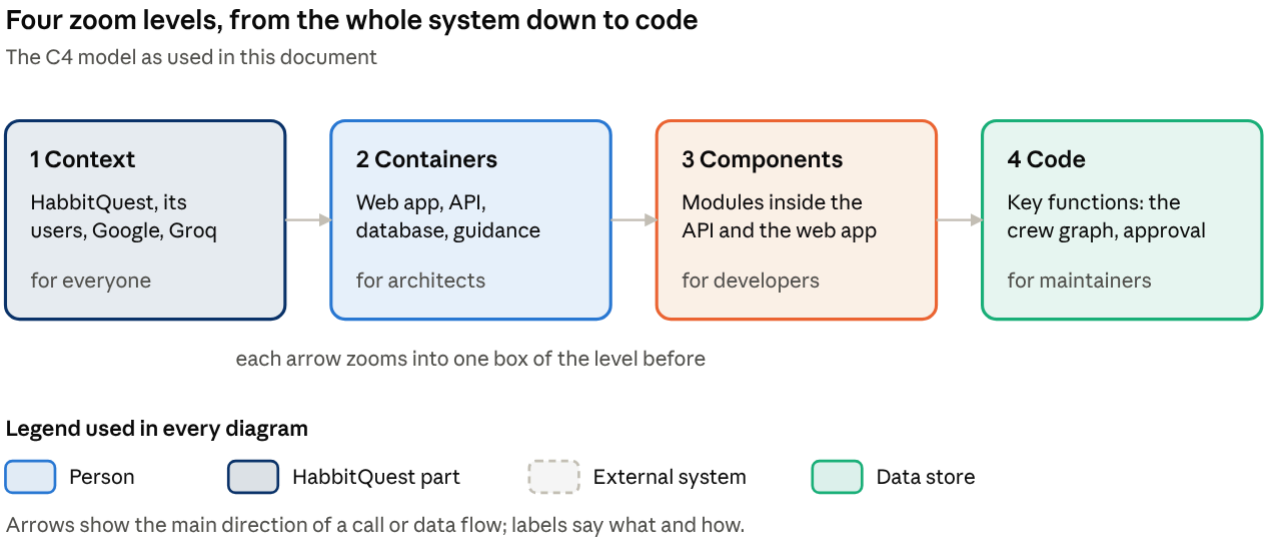
Doc5 — C4 Model Diagrams: HabbitQuest

Nov 3, 2026 · @Rahul

About the C4 model

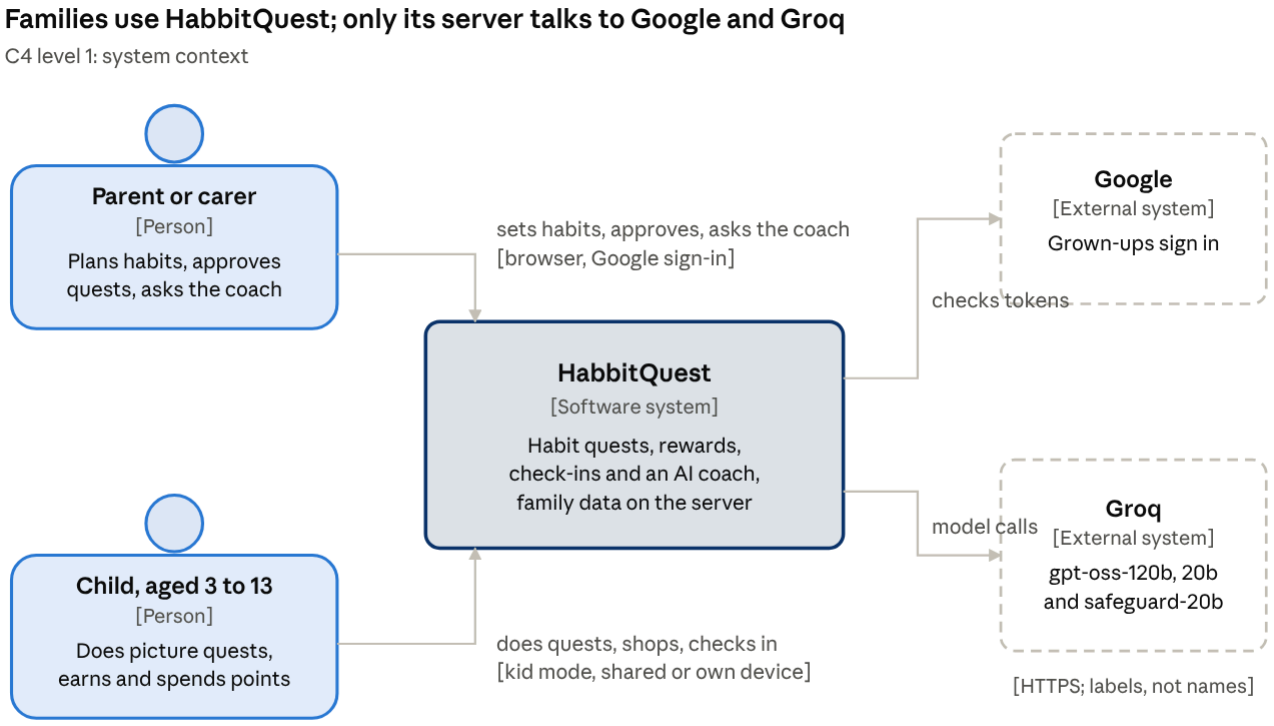
These diagrams describe HabbitQuest at four zoom levels, from the whole system to individual functions, so each reader can stop at the level they need.



C4 zoom levels and legend

All diagrams show the system as built (November 2026) unless marked "target".

Level 1: System context



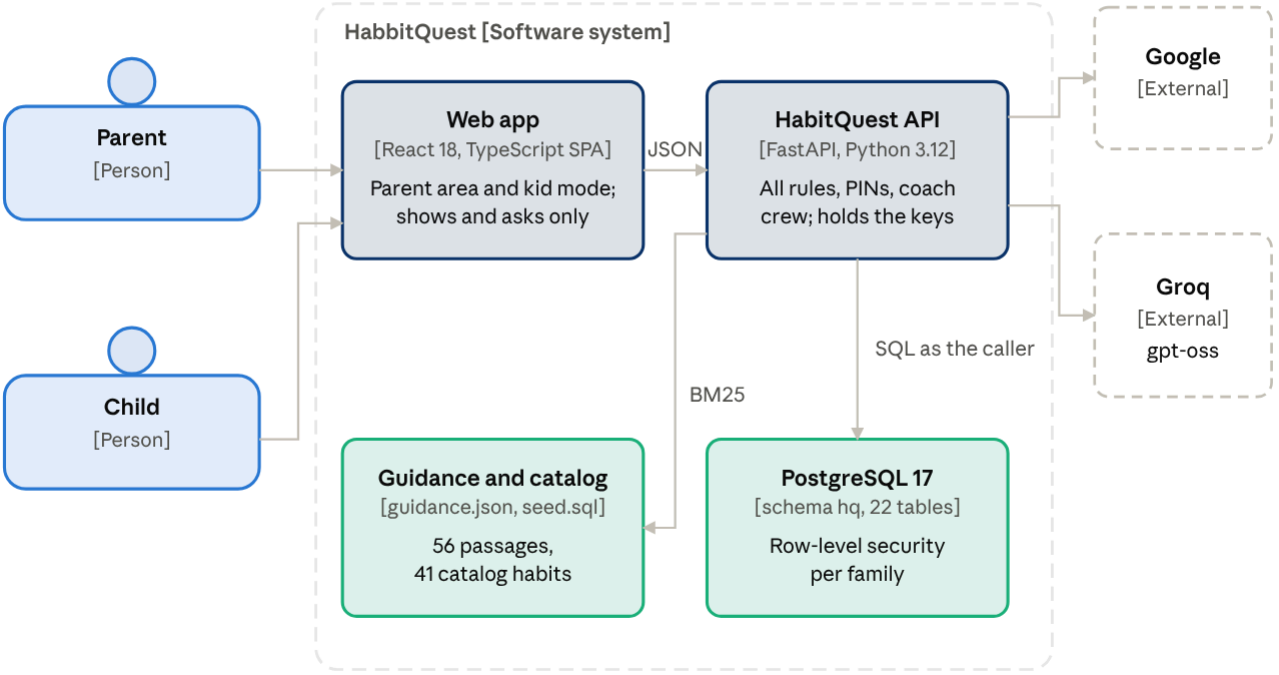
C4 level 1 · system context

HabbitQuest has two kinds of users, on a shared device or the child's own, and two external services: Google for sign-in and Groq for the AI models. Groq never sees names, emails or PINs.

Level 2: Containers

Four containers: web app, API, database and guidance

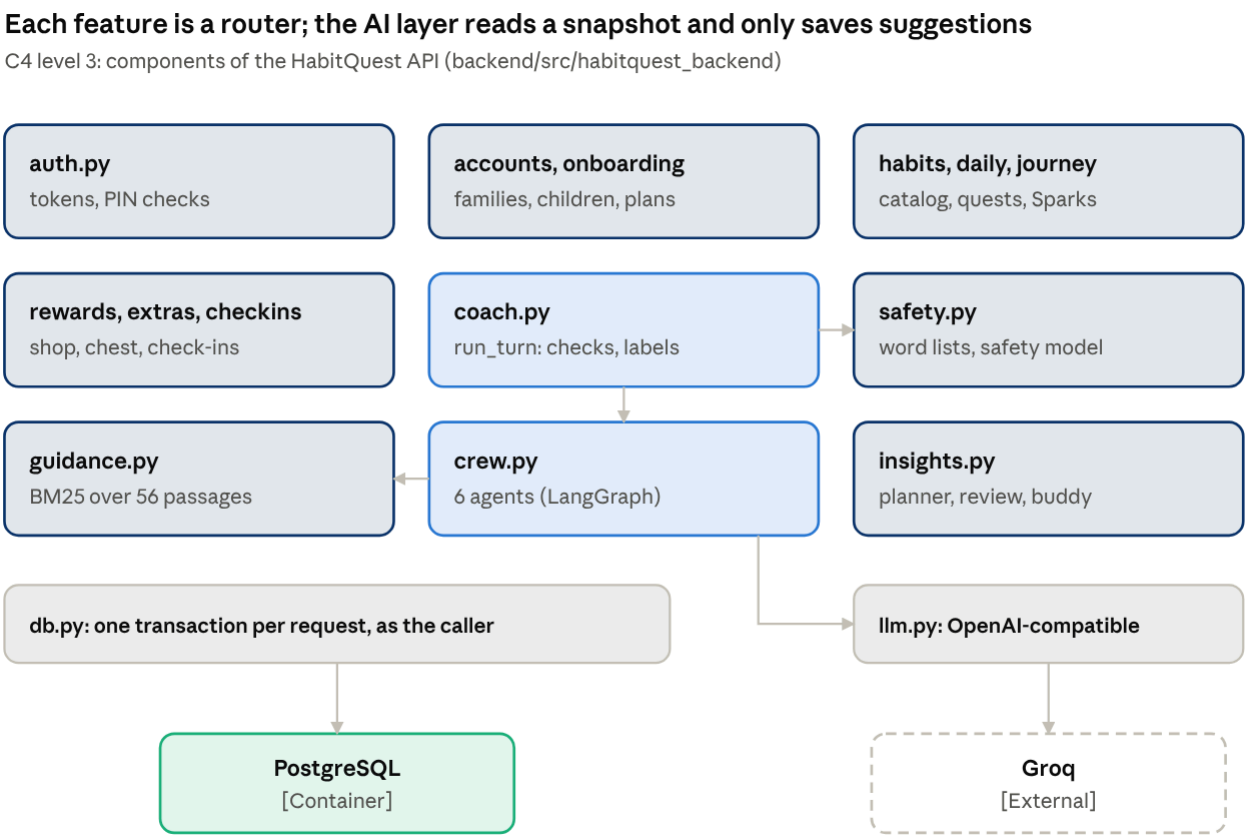
C4 level 2: containers



C4 level 2 · containers

The web app is the only container people touch. The API holds every secret and all family data; PostgreSQL's row-level security keeps each family's rows apart, and the AI helpers only save suggestions.

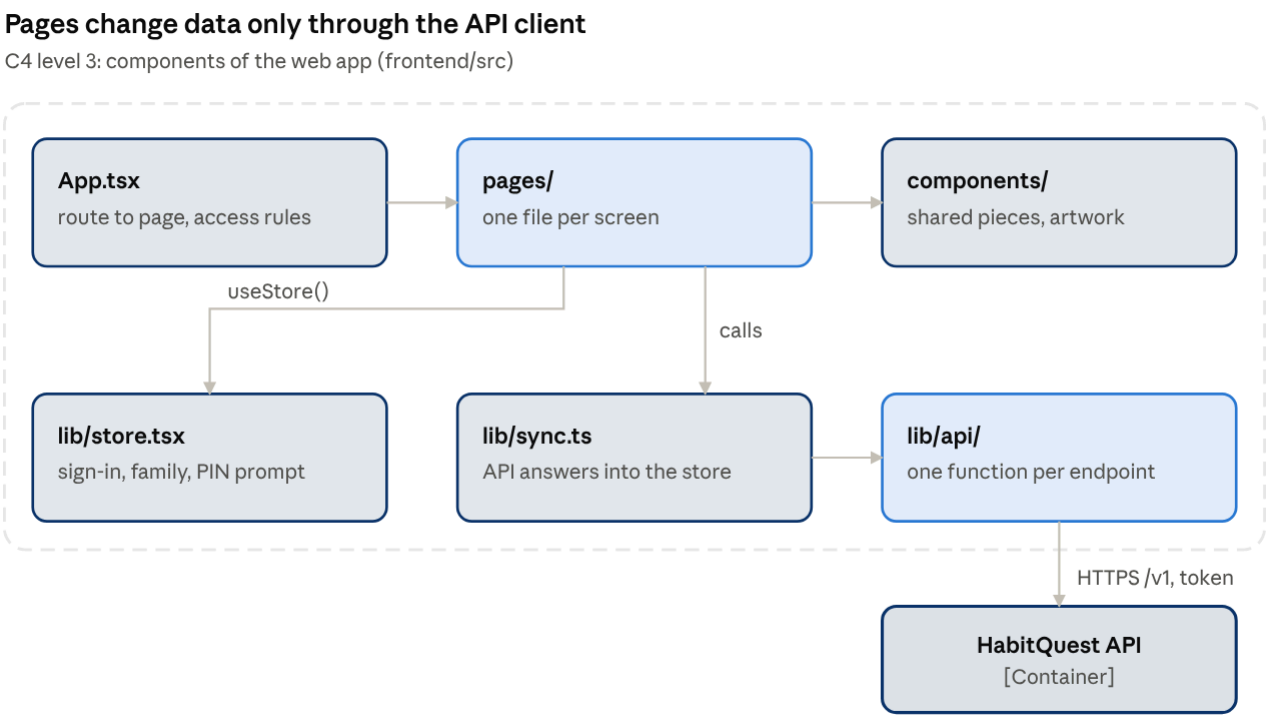
Level 3: Components of the API



C4 level 3 · API components

Each router runs its SQL in `db.transaction` as the caller. A coach turn (`coach.run_turn`) runs the risk checks, swaps names for labels and then runs the crew; nothing in the AI layer writes family data except saving suggestions for a grown-up to accept.

Level 3: Components of the web app



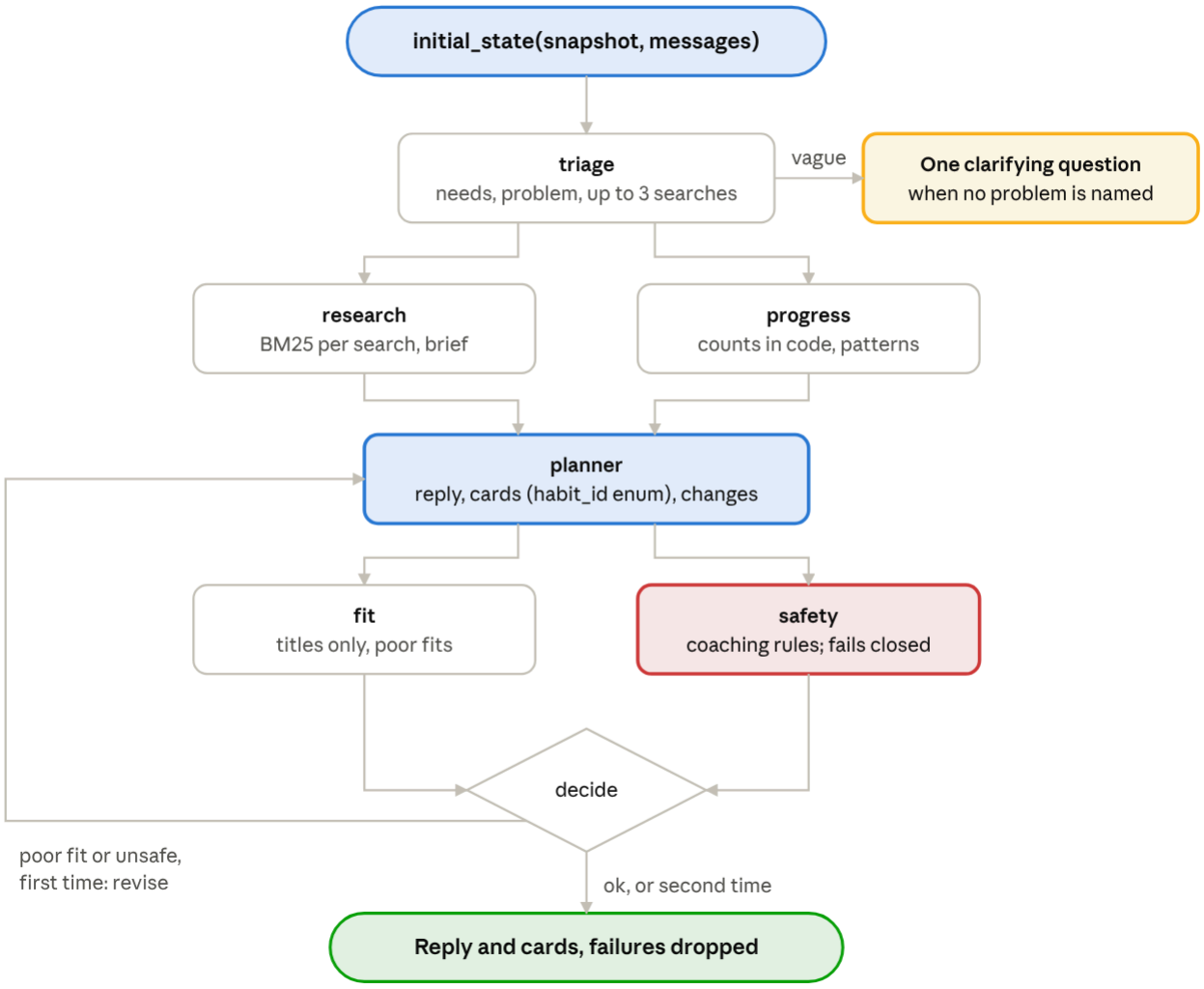
C4 level 3 · web app components

Pages read state with `useStore()` and change data only through `lib/api`; `lib/sync.ts` copies the API's answers back into the store. Kids' notes are screened on the server.

Level 4: Code view

The crew is a fixed graph: two pairs run side by side, one revision at most

C4 level 4: build_crew() in crew.py (LangGraph StateGraph)



C4 level 4 · the crew graph

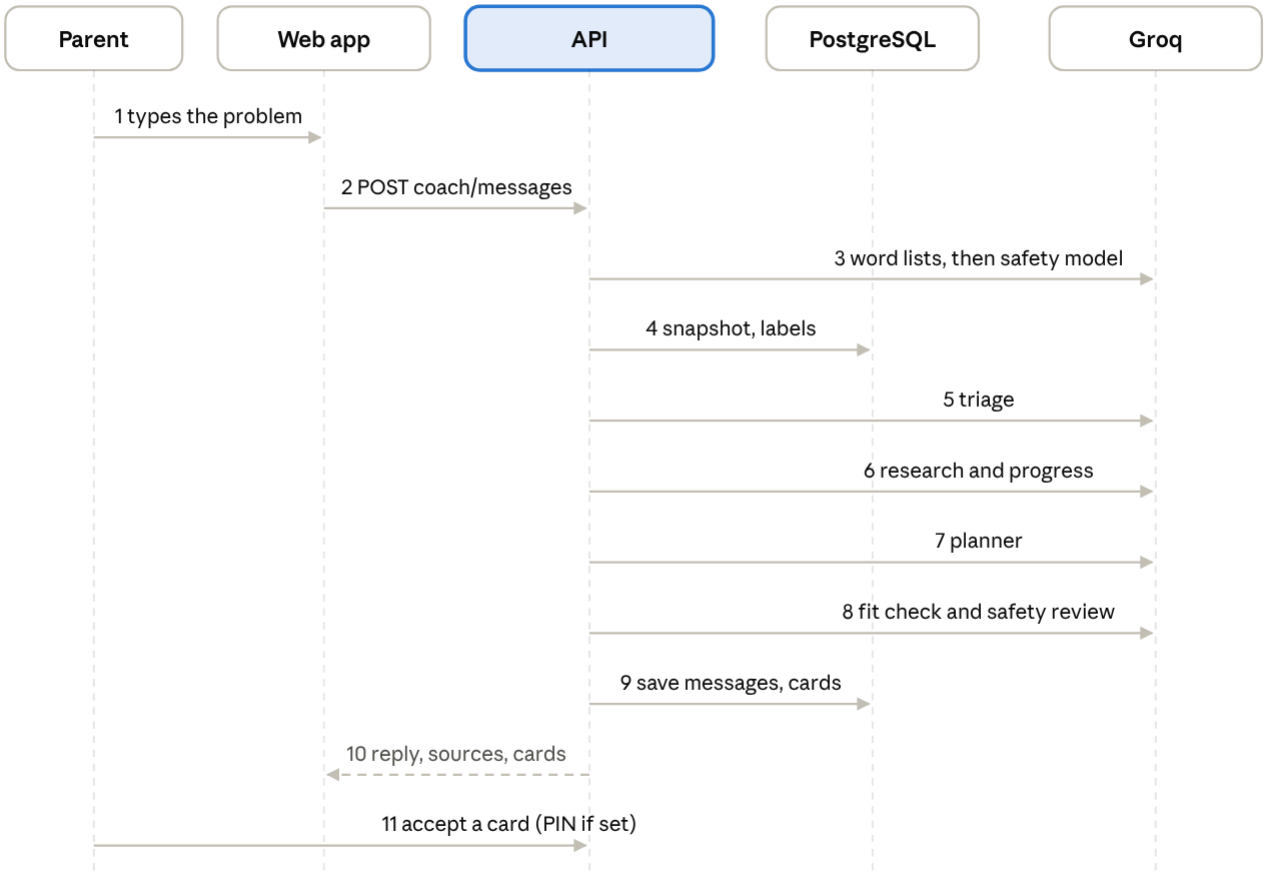
The planner can only name habits the code says fit: its schema's `habit_id` is an enum built for each turn. Quest approval is the other key function at this level, in `daily.py` (shortened):

```
def decide(user: UUID, completion_id: UUID, status: str) -> dict:
    with transaction(settings, user) as conn:          # runs as the caller:
row-level security applies
        row = one(conn, "SELECT ... FROM hq.completions c ... WHERE c.id=%s
FOR UPDATE OF c", (completion_id,))
        if row["status"] != "submitted":
            raise HTTPException(409, f"This completion was already
{row['status']}")  # pays once
        completion = one(conn, "UPDATE hq.completions SET status=%s, ...
RETURNING ...", (status, user, completion_id))
        if status == "approved":
            reasons = boosts(conn, row["child_id"],
row["day"]).get(row["habit_id"], [])
            conn.execute("INSERT INTO hq.points_ledger(...)
VALUES(%s,%s,'earned',%s,%s)",
                         (row["child_id"], row["points"] *
multiplier(reasons), completion_id, user))
        return {"completion": completion, "progress": progress(conn,
row["child_id"]),
                "points": balance(conn, row["child_id"])}
```

Dynamic view: a parent asks the habit coach

A parent asks the coach: eleven steps from message to accepted habit

Dynamic view; numbers match the list below

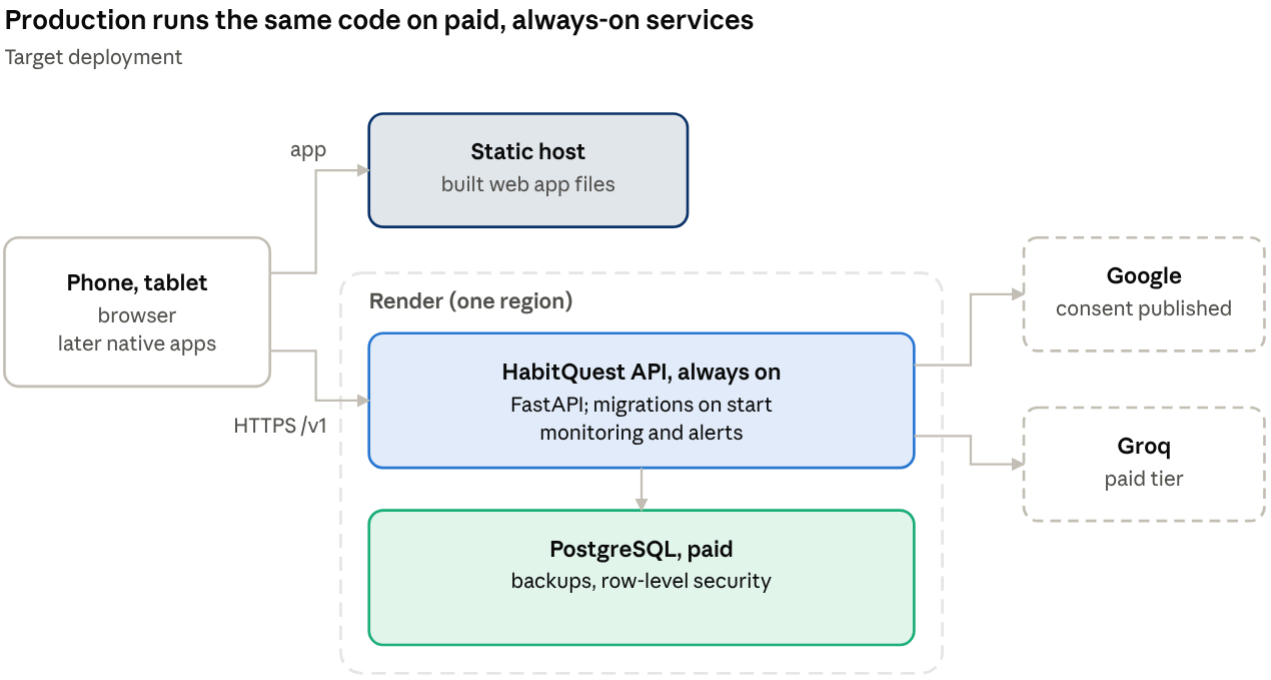


dynamic view · 11 numbered interactions

1. The parent types "Maya melts down when screen time ends" with Maya selected.
2. The web app sends `POST /v1/children/{id}/coach/messages` with the caregiver token.
3. The API checks the message with word lists, then the safety model; a risk ends the turn with crisis guidance.
4. It takes a snapshot of Maya's age, plan and progress from PostgreSQL, with her name replaced by "Child A".
5. Triage decides what the parent needs and writes up to 3 searches.
6. Research ranks guidance passages with BM25 while the progress analyst reads counts made in code; both run at once.
7. The planner writes the reply, up to 3 cards (only habits that fit Maya) and plan changes, citing passages as [G#].
8. The fit check and the safety reviewer read the result; a poor fit or an unsafe reply goes back to the planner once.

9. The API puts Maya's name back, saves the messages and suggestions, and returns them.
10. The web app shows the reply, its sources and the cards.
11. The parent accepts a card (with the PIN, if set); the API checks the plan rules again and adds the habit.

Target production deployment



target deployment · device, static host, Render, Google, Groq

Production runs the same code: the API as an always-on Render web service (any container host would do) with a paid PostgreSQL and backups, the built web app on a static host, Groq on a paid tier, and the Google consent screen published.